Week-8 servelts

JAVA SERVLET PROGRAMMING – LAB RECORD

Subject: Java Web Programming
Topic: Servlets and JDBC
Platform: Windows / Linux
Software: JDK, Apache Tomcat 10+, Eclipse IDE or IntelliJ IDEA, MySQL

EXPERIMENT 1: HTTP SERVLET USING GET AND POST

1. Aim

To develop a simple HTTP Servlet using the HttpServlet class and demonstrate handling HTTP GET and POST requests.

2. Requirements

- JDK
- Apache Tomcat 10+
- Eclipse IDE
- HTML

3. Procedure

1. Open Eclipse IDE.
2. Create a Dynamic Web Project named ServletDemo.
3. Select Apache Tomcat as the server.
4. Create a Java class named GetPostServlet.
5. Extend the HttpServlet class.
6. Override the doGet() and doPost() methods.
7. Add the servlet annotation.
8. Run the project on the Tomcat server.
9. Open the browser and enter the servlet URL.

GetPostServlet.java

java

```
import java.io.*;
import jakarta.servlet.*;
import jakarta.servlet.annotation.WebServlet;
import jakarta.servlet.http.*;

@WebServlet("/getpost")
public class GetPostServlet extends HttpServlet {

    protected void doGet(HttpServletRequest request,
            HttpServletResponse response)
            throws ServletException, IOException {

        response.setContentType("text/html");

        PrintWriter out = response.getWriter();
```

```
        out.println("<h2>This is HTTP GET Method</h2>");
    }

    protected void doPost(HttpServletRequest request,
            HttpServletResponse response)
            throws ServletException, IOException {

        response.setContentType("text/html");

        PrintWriter out = response.getWriter();

        out.println("<h2>This is HTTP POST Method</h2>");
    }
}
```

index.html

HTML

Wrap linesCopy codePreview

```
<html>
<body>
    <h2>HTTP Servlet Demo</h2>

    <form action="getpost" method="get">
        <input type="submit" value="GET Request">
    </form>

    <form action="getpost" method="post">
```

```
    <input type="submit" value="POST Request">
  </form>
</body>
</html>
```

5. Expected Output

When the GET button is clicked:

text

This is HTTP GET Method

When the POST button is clicked:

text

This is HTTP POST Method

6. Result

Thus, the HTTP Servlet was successfully developed and GET and POST requests were handled.

EXPERIMENT 2: GENERICSERVLET CLASS

1. Aim

To develop a servlet by extending the GenericServlet class.

2. Requirements

- JDK
- Apache Tomcat 10+
- Eclipse IDE

3. Procedure

1. Create a Dynamic Web Project named GenericServletDemo.
2. Create a Java class named MyGenericServlet.
3. Extend the GenericServlet class.
4. Override the service() method.
5. Set the response content type.

6. Display a message using PrintWriter.
7. Run the application using Tomcat.

4. Program

MyGenericServlet.java

java

```
import java.io.*;
import jakarta.servlet.*;
import jakarta.servlet.annotation.WebServlet;

@WebServlet("/generic")
public class MyGenericServlet extends GenericServlet {

    public void service(ServletRequest request,
            ServletResponse response)
            throws ServletException, IOException {

        response.setContentType("text/html");

        PrintWriter out = response.getWriter();

        out.println("<html>");
        out.println("<body>");
        out.println("<h2>Welcome to Generic Servlet</h2>");
        out.println("<h3>Servlet executed successfully</h3>");
        out.println("</body>");
        out.println("</html>");
    }
```

}

5. Expected Output

text

Welcome to Generic Servlet

Servlet executed successfully

6. Result

Thus, a servlet was successfully developed by extending the GenericServlet class.

EXPERIMENT 3: SERVLET INTERFACE AND LIFE CYCLE

1. Aim

To implement the Servlet interface and demonstrate the servlet life-cycle methods.

2. Requirements

- JDK
- Apache Tomcat 10+
- Eclipse IDE

3. Procedure

1. Create a Dynamic Web Project named ServletLifeCycle.
2. Create a Java class named LifeCycleServlet.
3. Implement the Servlet interface.
4. Override all five interface methods.
5. Implement the init(), service() and destroy() methods.
6. Display messages during servlet execution.
7. Run the servlet using Tomcat.

4. Program

LifeCycleServlet.java

java

```
import java.io.*;
import jakarta.servlet.*;
import jakarta.servlet.annotation.WebServlet;

@WebServlet("/life")
public class LifeCycleServlet implements Servlet {

   ServletConfig config;

   public void init(ServletConfig config)
         throws ServletException {

      this.config = config;
      System.out.println("Servlet Initialized");
   }

   public void service(ServletRequest request,
         ServletResponse response)
         throws ServletException, IOException {

      response.setContentType("text/html");

      PrintWriter out = response.getWriter();

      out.println("<h2>Servlet Life Cycle</h2>");
      out.println("<h3>Service Method Executed</h3>");
```

```
    }

    public void destroy() {
        System.out.println("Servlet Destroyed");
    }

    public ServletConfig getServletConfig() {
        return config;
    }

    public String getServletInfo() {
        return "Servlet Life Cycle Example";
    }
}
```

5. Expected Output

Browser Output:

text

Servlet Life Cycle

Service Method Executed

Tomcat Console:

text

Servlet Initialized

Servlet Destroyed

Note: The destroy message appears when the servlet is unloaded or the server is stopped.

6. Servlet Life Cycle

1. **init():** Called once when the servlet is initialized.

2. **service():** Called to process client requests.
3. **destroy():** Called before the servlet is removed.

The other two methods provide servlet configuration and information.

7. Result

Thus, the Servlet interface was successfully implemented and its life-cycle methods were demonstrated.

EXPERIMENT 4: HTML FORM PROCESSING USING JAVA SERVLET

1. Aim

To develop an HTML form and process its data using a Java Servlet.

2. Requirements

- JDK
- Apache Tomcat 10+
- Eclipse IDE
- HTML

3. Procedure

1. Create a Dynamic Web Project named FormDemo.
2. Create an HTML file named form.html.
3. Add name and email input fields.
4. Set the form method to POST.
5. Create a servlet named FormServlet.
6. Read the submitted values using request.getParameter().
7. Display the received data in the browser.
8. Run the project.

4. Program

form.html

HTML

Wrap linesCopy codePreview

```
<html>
<body>

<h2>Student Registration Form</h2>

<form action="submit" method="post">

   Name:
   <input type="text" name="name" required>
   <br><br>

   Email:
   <input type="email" name="email" required>
   <br><br>

   <input type="submit" value="Submit">

</form>

</body>
</html>
```

FormServlet.java

java

```
import java.io.*;
import jakarta.servlet.*;
import jakarta.servlet.annotation.WebServlet;
import jakarta.servlet.http.*;
```

```
@WebServlet("/submit")
public class FormServlet extends HttpServlet {

    protected void doPost(HttpServletRequest request,
            HttpServletResponse response)
            throws ServletException, IOException {

        response.setContentType("text/html");

        String name = request.getParameter("name");
        String email = request.getParameter("email");

        PrintWriter out = response.getWriter();

        out.println("<h2>Registration Successful</h2>");
        out.println("<p>Name: " + name + "</p>");
        out.println("<p>Email: " + email + "</p>");
    }
}
```

5. Expected Output

Input:

text

Name: Rahul

Email: rahul@gmail.com

Output:

text

Registration Successful

Name: Rahul

Email: rahul@gmail.com

6. Result

Thus, an HTML form was successfully created and its submitted data was processed using a Java Servlet.

EXPERIMENT 5: SERVLET CONNECTIVITY WITH MYSQL USING JDBC

1. Aim

To develop a servlet application that connects to a MySQL database using JDBC.

2. Requirements

- JDK
- Apache Tomcat 10+
- Eclipse IDE
- MySQL Server
- MySQL JDBC Driver

3. Procedure

Step 1: Create a Database

Open MySQL and execute:

sql

```
CREATE DATABASE studentdb;


USE studentdb;


CREATE TABLE student (
```

```
    id INT AUTO_INCREMENT PRIMARY KEY,
    name VARCHAR(50),
    email VARCHAR(100)
);
```

Step 2: Add MySQL JDBC Driver

Download MySQL Connector/J and add its JAR file to the project's library or WEB-INF/lib folder.

Step 3: Create a Java Servlet

Create a servlet named DatabaseServlet.

Step 4: Establish Database Connection

Use DriverManager.getConnection() to connect to MySQL.

Step 5: Execute SQL Query

Use PreparedStatement to insert student details.

Step 6: Run the Application

Start MySQL and Tomcat, then open the servlet URL.

4. Program

DatabaseServlet.java

java

```
import java.io.*;
import java.sql.*;
import jakarta.servlet.*;
import jakarta.servlet.annotation.WebServlet;
import jakarta.servlet.http.*;

@WebServlet("/database")
public class DatabaseServlet extends HttpServlet {

    protected void doGet(HttpServletRequest request,
```

```
    HttpServletResponse response)
    throws ServletException, IOException {

response.setContentType("text/html");

PrintWriter out = response.getWriter();

String url =
    "jdbc:mysql://localhost:3306/studentdb";

String user = "root";
String password = "your_password";

String sql =
    "INSERT INTO student(name, email) VALUES (?, ?)";

try {
    Class.forName("com.mysql.cj.jdbc.Driver");

    Connection con =
        DriverManager.getConnection(
            url, user, password);

    PreparedStatement ps =
        con.prepareStatement(sql);

    ps.setString(1, "Rahul");
```

```
            ps.setString(2, "rahul@gmail.com");

            int result = ps.executeUpdate();

            if (result > 0) {
                out.println("<h2>Record Inserted Successfully</h2>");
            }

            ps.close();
            con.close();

        } catch (Exception e) {
            out.println("<h2>Database Error</h2>");
            out.println("<p>" + e.getMessage() + "</p>");
        }
    }
}
```

Replace your_password with your actual MySQL password.

5. Expected Output

text

Record Inserted Successfully

MySQL Table:

id name email

1 Rahul rahul@gmail.com

6. Result

Thus, the Java Servlet was successfully connected to the MySQL database using JDBC and student information was inserted.

EXPERIMENT 6: ONLINE ORDER PROCESSING APPLICATION

1. Aim

To develop an online order processing application using HTML and Java Servlet.

2. Requirements

- JDK
- Apache Tomcat 10+
- Eclipse IDE
- HTML

3. Procedure

1. Create a Dynamic Web Project named OrderApplication.
2. Create an HTML page named order.html.
3. Add customer name, product and quantity fields.
4. Create a servlet named OrderServlet.
5. Read the customer order details.
6. Calculate the total price.
7. Display the order summary.
8. Run the application using Tomcat.

4. Program

order.html

HTML

Wrap linesCopy codePreview

<html>

<body>

<h2>Online Order Form</h2>

```
<form action="order" method="post">

    Customer Name:
    <input type="text" name="name" required>
    <br><br>

    Select Product:

    <select name="product">
        <option value="Laptop">Laptop - 50000</option>
        <option value="Mobile">Mobile - 20000</option>
        <option value="Headphones">Headphones - 2000</option>
    </select>

    <br><br>

    Quantity:
    <input type="number" name="quantity"
           min="1" required>

    <br><br>

    <input type="submit" value="Place Order">

</form>

</body>
```

```
</html>
```

OrderServlet.java

java

```
import java.io.*;
import jakarta.servlet.*;
import jakarta.servlet.annotation.WebServlet;
import jakarta.servlet.http.*;

@WebServlet("/order")
public class OrderServlet extends HttpServlet {

    protected void doPost(HttpServletRequest request,
            HttpServletResponse response)
            throws ServletException, IOException {

        response.setContentType("text/html");

        PrintWriter out = response.getWriter();

        String name = request.getParameter("name");
        String product = request.getParameter("product");

        int quantity =
            Integer.parseInt(
                request.getParameter("quantity"));

        int price = 0;
```

```
        if (product.equals("Laptop")) {
            price = 50000;
        } else if (product.equals("Mobile")) {
            price = 20000;
        } else if (product.equals("Headphones")) {
            price = 2000;
        }

        int total = price * quantity;

        out.println("<h2>Order Placed Successfully</h2>");

        out.println("<p>Customer: " + name + "</p>");
        out.println("<p>Product: " + product + "</p>");
        out.println("<p>Quantity: " + quantity + "</p>");
        out.println("<p>Price: Rs. " + price + "</p>");
        out.println("<h3>Total: Rs. " + total + "</h3>");
    }
}
```

5. Expected Output

Input:

text

Customer Name: Rahul

Product: Laptop

Quantity: 2

Output:

text

Order Placed Successfully

Customer: Rahul

Product: Laptop

Quantity: 2

Price: Rs. 50000

Total: Rs. 100000

6. Result

Thus, an online order processing application was successfully developed using HTML and Java Servlet.

EXPERIMENT 7: READING CLIENT DATA AND SENDING DYNAMIC RESPONSE

1. Aim

To develop a servlet that reads data submitted by a client and sends a dynamic response back to the client.

2. Requirements

- JDK
- Apache Tomcat 10+
- Eclipse IDE
- HTML

3. Procedure

1. Create a Dynamic Web Project named DynamicResponse.
2. Create an HTML file named input.html.
3. Add a text field to accept the client's name.
4. Create a servlet named DynamicServlet.

5. Read the client data using request.getParameter().
6. Generate a dynamic HTML response.
7. Run the application using Tomcat.
8. Enter a name and click Submit.

4. Program

input.html

HTML

Wrap linesCopy codePreview

```
<html>
<body>

<h2>Enter Your Details</h2>

<form action="dynamic" method="post">

   Enter Your Name:

   <input type="text" name="username" required>

   <br><br>

   <input type="submit" value="Submit">

</form>

</body>
</html>
```

DynamicServlet.java

java

```
import java.io.*;
import jakarta.servlet.*;
import jakarta.servlet.annotation.WebServlet;
import jakarta.servlet.http.*;

@WebServlet("/dynamic")
public class DynamicServlet extends HttpServlet {

    protected void doPost(HttpServletRequest request,
            HttpServletResponse response)
            throws ServletException, IOException {

        response.setContentType("text/html");

        String name =
            request.getParameter("username");

        PrintWriter out = response.getWriter();

        out.println("<html>");
        out.println("<body>");

        out.println("<h2>Welcome " + escape(name) + "!</h2>");

        out.println("<h3>Your request was processed successfully.</h3>");
```

```
        out.println("</body>");
        out.println("</html>");
    }

    private String escape(String value) {
        return value.replace("&", "&amp;")
                    .replace("<", "&lt;")
                    .replace(">", "&gt;")
                    .replace("\"", "&quot;")
                    .replace("'", "&#39;");
    }
}
```

5. Expected Output

Input:

text

Enter Your Name: Rahul

Output:

text

Welcome Rahul!

Your request was processed successfully.

6. Result

Thus, a Java Servlet was successfully developed to read client-submitted data and generate a dynamic response.

COMMON EXECUTION STEPS

Follow these steps to execute all seven experiments.

1. Install JDK on your computer.
2. Install Eclipse IDE for Enterprise Java and Web Developers.
3. Install Apache Tomcat 10 or later.
4. Open Eclipse IDE.
5. Select File → New → Dynamic Web Project.
6. Enter the project name.
7. Select the configured Tomcat server.
8. Create the required Java Servlet class.
9. Add the appropriate HTML file.
10. Write and save the programs.
11. Right-click the project.
12. Select Run As → Run on Server.
13. Select Apache Tomcat.
14. Open the application in your browser.
15. Check the expected output.

Important compatibility note: These programs use jakarta.servlet imports and are intended for Tomcat 10+. For Tomcat 9, use the older javax.servlet imports instead.

OVERALL RESULT

Thus, all seven Java Servlet experiments were studied and implemented successfully. The experiments demonstrated HTTP GET and POST methods, GenericServlet, Servlet life-cycle methods, HTML form processing, JDBC database connectivity, online order processing and dynamic client responses.